

CS & IT ENGINEERING



Algorithms

Analysis of Algorithms

Lecture No.- 11



By- Aditya sir

Recap of Previous Lecture



Topic

Recursive Algo Time Complexity Analysis



Topics to be Covered



Topic

Time Complexity of Recursive Algo (New Type)

Topic

Loop Complexity

ABOUT ME

Hello, I'm Aditya.

1. Represented college as the first Google DSC Ambassador.
2. The only student from the batch to secure an internship at Amazon.
3. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt
- City topper
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10. Joined Dream11 as a Data Scientist.
9. Have mentored around 10k working professionals in field of Data Science and Analytics
10. Have been mentoring GATE aspirants to secure a great rank in limited time.
11. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.



By- Aditya sir

[MCQ]

#Q. Algo $AJ(n)$ {
 if $(n == 1)$ return 1
 else return $(AJ(n/2) + AJ(n))$
}

37%

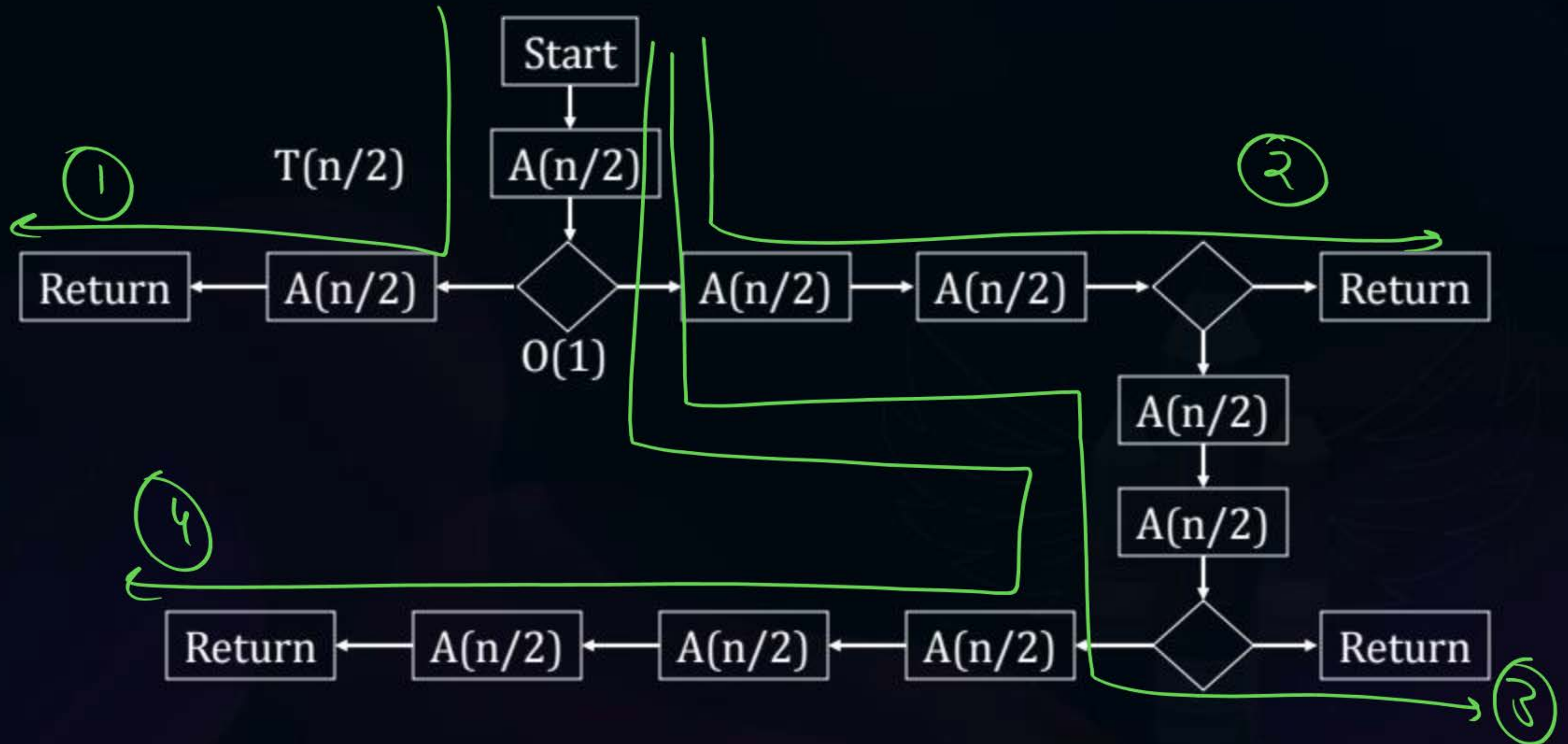
 $\downarrow$
 $O(n)$ ☒ **A** $O(n)$ **A**☐ **B** $O(n \log n)$ ☐ **C** $O(\log n)$ ☐ **D** $O(n^2)$

#Q. The given diagram represents the flowchart of recursive algorithm $A(n)$. Assume that all statement except for the recursive calls have order (1) time complexity. Then the best case and worst case time of this algorithm is _____.

- ① Best Case $\rightarrow O(\quad)$
- ② Worst Case $\rightarrow O(\quad)$



Topic : Time Complexity Framework for Recursive Algorithms





Topic : Time Complexity Framework for Recursive Algorithms

Best Case:

- Case when algorithm takes min steps/does min work/effort.

Worst Case:

- Case when algorithm takes max steps/work/effort.



Topic : Time Complexity Framework for Recursive Algorithms

Case 1:

- a) ✓ 2 RC $\rightarrow$ B.C
- b) ✓ 3 RC
- c) ✓ 5 RC
- d) ✓ 8 RC $\rightarrow$ W.C



Topic : Time Complexity Framework for Recursive Algorithms

Case 1: Best Case Analysis:

Step 1: Recurrence

$$T(n) = 2T(n/2) + a, n > 1$$

$$T(n) = b, \underline{n = 1}$$



Topic : Time Complexity Framework for Recursive Algorithms

$$T(n) = 2T(n/2) + a$$

$$T(n/2) = 2T(n/2^2) + a$$

$$T(n) = 2[2T(n/2^2) + a] + a = 2^2 T(n/2^2) + 3a$$

$$T(n) = 2^2 T(n/2^2) + (2^2 - 1) a$$

$$T(n) = 2^3 T(n/2^3) + (2^3 - 1) a$$

.

.

$$T(n) = 2^k T(n/2^k) + (2^k - 1) a$$



Topic : Time Complexity Framework for Recursive Algorithms

For BC,

$$n/2^k = 1$$

$$2^k = n$$

$$T(n) = n * T(1) + (n - 1) a$$

$$= n * b + an - a$$

$$= O(n)$$



Topic : Time Complexity Framework for Recursive Algorithms

Case 1: Worst Case Analysis:

Step 1: $T(n) = 8T(n/2) + a, n > 1$

$$T(n) = b, n = 1$$



Topic : Time Complexity Framework for Recursive Algorithms

$$T(n) = 8T(n/2) + a$$

$$T(n/2) = 8T(n/2^2) + a$$

$$T(n) = 8[8T(n/2^2) + a] + a = 8^2 T(n/2^2) + 9a$$

$$T(n) = 8^3 T(n/2^3) + 8^3 a + 9a$$

$$T(n) = 8^3 T(n/2^3) + (8^2 + 8^1 + 8^0)a$$

.

.

$$T(n) = 8^k T(n/2^k) + (8^{k-1} + 8^{k-2} + \dots + 8^0)a$$



Topic : Time Complexity Framework for Recursive Algorithms

$$T(n) = 8^k T(n/2^k) + (8^{k-1} + 8^{k-2} + \dots + 8^0)a$$

$$\text{GP: } a = 1, r = 8, n = k$$

$$\frac{a(8^n - 1)}{8 - 1} = \frac{1(8^k - 1)}{8 - 1} = \left[\frac{8^k - 1}{7} \right]$$

$$n/2^k = 1$$

$$\boxed{2^k = n} \Rightarrow 8^k = n^3$$

$$T(n) = 8^k T(1) + \left(\frac{8^k - 1}{7} \right) a$$

$$\text{Put } 8^k = n^3$$

$$T(n) = n^3 * b + \left(\frac{n^3 - 1}{7} \right) a \Rightarrow \boxed{O(n^3)} \checkmark$$

[MCQ]

#Q. Given:

$$T(n) = 2T(n/2) + n \log n, n > 1$$

$$T(n) = 1, n=1$$

53%

A $O(n \log n)$ **B** $O(n^2)$ ☒ **C** $O(n * (\log n)^2)$ **D** $O(n)$

$$T(n) = 2T(n/2) + n \log n$$

$$T(n/2) = 2T(n/2^2) + \frac{n}{2} \log(n/2)$$

$$T(n) = 2^2 T(n/2^2) + n \log n/2 + n \log n$$

$$T(n) = 2^3 T(n/2^3) + n \log(n/2^2) + n \log(n/2) + n \log(n/2^0)$$

$$= 2^3 T(n/2^3) + n \left[\log(n/2^2) + \log(n/2^1) + \log(n/2^0) \right]$$

general,

$$T(n) = 2^k T(n/2^k) + n \left[\log(n/2^0) + \log(n/2^1) + \dots + \log\left(\frac{n}{2^{k-1}}\right) \right]$$

$$\boxed{\log a + \log b = \log(ab)}$$

$$\Rightarrow \log\left(\frac{n}{2^0} * \frac{n}{2^1} * \frac{n}{2^2} * \dots * \frac{n}{2^{k-1}}\right)$$

$$2^0 \times 2^1 \times 2^2 \times \dots \times 2^{k-1}$$
$$= 2^{[0+1+2+\dots+(k-1)]}$$

$$\left[0+1+2+\dots+(k-1) = \frac{k(k-1)}{2} \right]$$

$$\boxed{2^{\frac{k(k-1)}{2}}} \Rightarrow \log\left(\frac{n^k}{2^{\frac{k(k-1)}{2}}}\right)$$

$$\begin{aligned} T(n) &= 2^k T(n/2^k) + n \times \log\left(\frac{n^k}{2^{\frac{k(k-1)}{2}}}\right) \\ &= 2^k T(n/2^k) + n \left(\log n^k - \log\left(2^{\frac{k(k-1)}{2}}\right) \right) \end{aligned}$$

$$= 2^k T(n/2^k) + k \times n \log n - \frac{k(k-1)}{2} \times n$$

For B.C,

$$n/2^k = 1$$

$$\boxed{2^k = n}$$

$$= n \times a + \underbrace{n \times (\log n)^2}_{\text{circled}} - \frac{n(\log n)^2 + n \log n}{2}$$



$$\boxed{T(n) = O(n(\log n)^2)}$$

Loop Complexities



Topic : Loop Complexity

Types of Loops:

(1) For; (2) While



Topic : Loop Complexity

For Loop:

```
for (i = 1; i <= n; i++)
```

```
{
```

```
    S1
```

```
    S2
```

$\rightarrow O(n)$

```
    S3
```

```
}
```



Topic : Loop Complexity

While Loop:

```
i = 1
```

```
while (i <= n)
```

```
{
```

```
    S1
```

```
    S2
```

```
    S3
```

```
    i++
```

```
}
```



Topic : Loop Complexity

How to determine the Time Complexity of a Loop:

Time Complexity of any loop depends on two important factors:

- (1) The number of times the loop is running/iterating/repeated.
- (2) The complexity of all the individual statements within it (inside loop body)



Topic : Loop Complexity

Example 1:

for (j = 1; j <= n; j++) $\longrightarrow$ runs n times

{

a = a + 5 $\longrightarrow O(1)$

}

$$\left. \begin{array}{l} j=1 \rightarrow O(1) \\ j=2 \rightarrow O(1) \\ \vdots \\ j=n \rightarrow O(1) \end{array} \right\} \Rightarrow n \times O(1) = \boxed{O(n)}$$



Topic : Loop Complexity

Example 2:

for (i = 1; i <= n/2; i++) $\longrightarrow$ $n/2$ times

{

 a = a + 10 $\longrightarrow$ $O(1)$

}

$$TC: O(n/2) = \underline{\underline{O(n)}}$$



Topic : Loop Complexity

Example 3:

```
for (i = 1; i <= n; i++)
```

```
{
```

```
    a = a + 3
```

```
    break;
```

```
}
```

→ O(1)



Topic : Loop Complexity

Example 4:

```
a = 0;  
for (i = 1; i <= n; i++)  
{  
    a = a + 5  
}
```

- Q1. What is the TC of code? $\rightarrow O(n)$
- Q2. What is the value of 'a' after code ends?

$$\rightarrow \underline{a = 5 * n}$$



Topic : Loop Complexity

Example 5:

V.IMP

```
a = 0;  
for (i = 1; i <= n; i++)  
{  
    a = a + i  
}  
print(a)
```

Q1. What is the TC of code?

→ $O(n)$

Q2. What is the output of given code?

→
$$a = \frac{n(n+1)}{2}$$



Topic : Loop Complexity

Example 6:

Algo AJ(x, n)

```
{  
    for(i = 1; i <= n; i++)  
    {  
        if (x % i == 0) {  
            break  
        }  
    }  
}
```



$x \% 1 = 0$ always $O(1)$

Q1. What is the best and worst case complexity of AJ() and for what type of input?



Topic : Loop Complexity

Example 7:

Algo AJ(x, n)

```
{  
    for(i = 2; i <= n; i++)  
    {  
        if (x % i == 0) {  
            break  
        }  
    }  
}
```

any even $x \rightarrow B.C \rightarrow O(1)$
 $x \rightarrow \text{prime no} > n \rightarrow W.C \rightarrow O(n)$

Q1. What is the time complexity of AJ()?

eg: $n=10, x=13$



Topic : Loop Complexity

Example 8:

Algo AJ(n)

```
{  
    for(i = 1; i <= n; i++)  
    {  
        AJ2(n)  
    }  
}
```

Q1. What is the time complexity of AJ0?

$$\Rightarrow \underline{O(n * O(AJ2(n)))}$$



Topic : Loop Complexity

While Loop:

```
i = 1           // initialization
while (i <= n)   // condition
{
    print(i)
    i++          // updation
}
```


[MCQ]

#Q. $i = 1, a = 0$ while ($i = 2$)

{

 $a = a + 1$

}

 $\text{while}()$
 $\{$
 $\} =$ $\text{while}(\text{True})$
 $\{$
 $\} =$

true statement

(infinite loop) $O(n)$  $O(1)$  $O(n \log n)$ 

none

 (∞)

Hw

- #Q. (i) What is the time complexity?
(ii) What is the value of a after code ends?

a = 0

```
for (i = 1; i <= n; i++) {  
    j = 1  
    while (j <= n/2) {  
        a = a + 5  
        j = j + 1  
    }  
}
```

nested loop

dependant independent

[NAT]

- #Q. (i) What is the time complexity?
(ii) What is the value of a after code ends?

a = 0

```
for (i = 1; i <= n; i += 5) {  
    a = a + 3  
}
```

Hw

→ Bonus



THANK - YOU